

1 The contract, adopted and not forked

The plugin's full tree: one display/ per page, one folder per unit, every unit the same shape.

```
<page>/
  <page>.md          the page, on stage
  display/          the plugin . discovery never enters
    <PageID>-Display1-<slug>/    ONE rendered unit
      README.md      claim . kind . caption-job . fragility . status
      intake/        caller-owned, provenance-bound values
        manifest.yaml
        inputs/      small approved summary CSV or JSON extracts only
      recipe/        renderer-owned script, spec, receipts
      float.tex      caption + label + asset reference
      preview.tex    standalone wrapper that inputs float.tex
      preview.pdf    compiled look, what the tab frames
      assets/        the WINNING render
      candidates/    pre-decision renders
      versions/      superseded variants, kept for history
    <PageID>-Display2-<slug>/    the next unit, same shape
```

The layout above is the display family's own contract file, `display-unit-output-contract.md`, copied as it stands rather than redesigned. The folders split into two halves: `intake/` and `recipe/` are the SOURCES, written by people and by drawing skills, while `assets/` and `preview.pdf` are BUILT from those sources, so a rebuild may overwrite the built half but never the sources. The only page-side change is the address: the unit sits inside the page's own `display/` folder, so when the page moves, its figures move with it.

2 The surface: one card per unit in the right pane

The tab: what a reader sees when they open Display on a page.

```
right pane . Display
+-----+
| QPf5-Display1-pipeline-tikz          |
|   preview.pdf, framed (body.py ccpdf) |
|   README status . rebuild            |
|   accepted: [ ] . shown, NOT tickable |
+-----+
one card per unit . empty display/ = empty state, not an error
```

The tab is built the same way the Slides tab is: a small button in the page's toolbar asks the server for this page's units, and the server writes a view that frames each unit's `preview.pdf`. `rebuild` redraws the built half from the sources; it never changes the inputs, and it never changes an approval. A unit is EVIDENCE: when a sentence names a unit's id in plain text, the id becomes a small button in that sentence, and clicking it opens the unit's card with its picture and its approval state. There is a second way to attach the same evidence, for machines: a `> Display:` line written UNDER a sentence, beside the `> Citation:` and `> Value:` lines, so a program can add evidence

without ever editing a person’s prose. Citing a unit that has no render yet is allowed with either form: the button then says a render is still owed. The exports keep the evidence too: the LaTeX PDF prints a cited unit as a real figure after the paragraph that cites it, and the Word file embeds the picture with its caption and adds a comment note on the sentence.

3 The writers, and the row no machine may tick

Who writes what: three hands on one unit, each confined to its half.

```

renderer skills    haipipe-display-table . -figure . -diagram . -illustration
                  write recipe/ . assets/ . preview.pdf
a person          rules intake/ and ticks accepted:
the pane          shows everything . writes NOTHING

```

The renderers are the display family’s existing drawing skills; this page adds none of its own. The approval row is the one rule with teeth: a [x] on `accepted:` means a person looked at one specific picture and said yes. A machine that ticks it has forged a judgment, not saved time, so no machine may.

The kinds and who draws them: one door routes to four renderers, plus one method with no skill.

```

haipipe-display . the DOOR . say the ask, it picks the renderer
data-driven . needs a ruled intake/ of numbers
  -table      aggregated CSV/JSON > booktabs LaTeX . recipe = the spec
  -figure     results > plot . recipe = the python script
concept-driven . the intake is a spec, not numbers
  -diagram    FigureSpec JSON > deterministic editable SVG
  tikz        .tikz.tex recipe > vector PDF . TeX-native: the paper's
              fonts, math labels, and float.tex may input it directly
              NO SKILL: authored by hand, as both units on this page were
  -illustration image-gen prompt > concept figure . recipe = prompt + receipts

```

A data-driven renderer draws numbers, and it may not go back to the raw data: it reads only the small approved extract in `intake/`. A concept-driven renderer draws an idea, so it carries no numbers at all; its input is the sketch it draws. Rebuild follows the kind: a python figure reruns its script, a diagram redraws from its spec, and an AI illustration replays its saved prompt.

4 How a unit is generated

The pipeline: five steps from approved numbers to an accepted render, and who moves each one.

- (1) INTAKE the caller fills `intake/`: `manifest.yaml` + small approved extracts, every value provenance-bound
- (2) RENDER the matching renderer skill runs, dispatched through `task-for-display`; it fills `recipe/` + `candidates/`
- (3) PICK a person picks the winner; it moves to `assets/`, the losers stay in `candidates/` or `versions/`
- (4) BUILD `preview.tex` compiles to `preview.pdf`; the adapter's

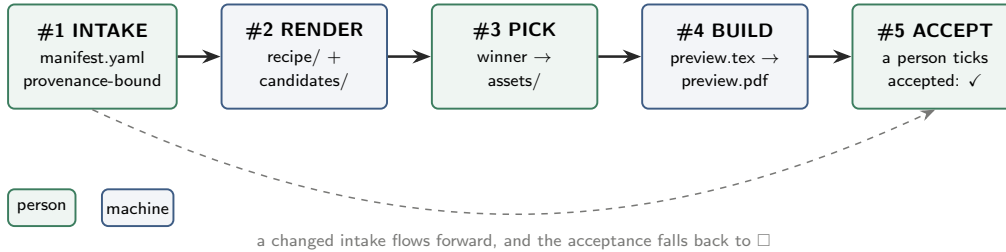


Figure 1: The display pipeline: five steps from approved intake to an accepted render. Green steps belong to a person, blue steps to machinery, and a changed intake returns the acceptance to open.

```

--check reports a stale unit instead of hiding it
(5) ACCEPT      a person reads the render and ticks accepted:

```

Drawing never comes first: with no approved inputs there is nothing safe to draw, and the intake contract exists to refuse exactly that. Steps (1) (3) (5) belong to a person and steps (2) (4) to machinery, the split of hands QPf5-Display1 draws; the tab's rebuild is step (4) alone, run again. If the inputs change, the work flows forward again from (2) to (5), and the old approval drops back to [], because the picture a person said yes to no longer exists.

Where (1)'s numbers come from: the evidence chain behind an intake, id-bound at every hop.

```

the page needs a number    > raised as a PROBE . the page's own probe/ plugin (QPf9)
probe orchestrator         > sweeps the bank . reuse | enrich | fresh
haipipe-task runs          > answer lands in <task-folder>/QA/<n>-<slug>.md
                           + the run's artifact, e.g. counts.csv . CANONICAL
intake/manifest.yaml       > cites that holder BY ID . carries only a
                           small approved extract in inputs/

```

A probe is a question card: the page asks its question once, the answer lands in the shared task bank, and from then on everyone cites the card's id instead of asking again. `intake/` is therefore not a second copy of the data: the task's own output stays the single source, and the manifest only points at it, by id. A unit names its evidence by id, never by a file path, and never types a number with its own hands; QPf5-Display2 draws this ask-once-cite-twice move. The slides plugin already works this way on QPf3: its deck asks for an id, a resolver turns the id into the file's real path, and the template holds no digits, so a hand-typed number has nowhere to live. Aim A1 asks for that same id lookup to work for units that live under a page.

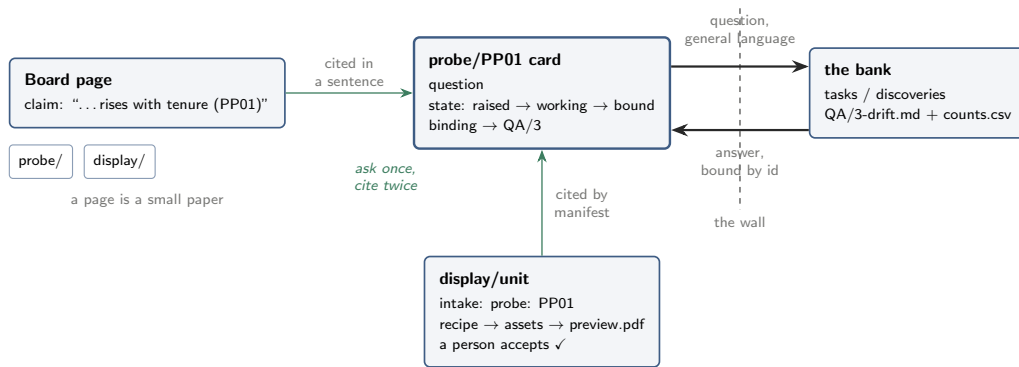


Figure 2: A page as a small paper: the probe plugin asks a question once, the bank answers behind the wall, and both the display intake and the page's own sentence cite the same card by id.